

Лабораторна робота. Варіант 4: швидкодія std::count_if

Виконав: Дмитро Демчук, група К-25

1. Методика

- **Збірка.** GCC 16 (g++-16), C++20; політики std::execution працюють поверх oneTBB (Apple Clang їх не підтримує). make run збирає програму з -O0 та -O3 і записує вивід у results/. Тестова машина: hardware_concurrency = 8.
- **Дані.** Цілі числа з [2, 100000] (mt19937, seed 42), три обсяги N = 100 000; 1 000 000; 10 000 000. Файли data/seq_N.txt: перший рядок - N, другий - N чисел через пробіл. Усі режими й обидві збірки читають однакові файли; якщо їх немає, програма створює їх сама, параметр --new створює нові.
- **Предикати.** isEvenNumber ($x \% 2 == 0$) - дешевий; isPrime (пробні ділення до $\sqrt{x}$) - дорогий.
- **Таймери.** total - читання файлу, count_if і вивід результату; compute - лише count_if на даних у RAM.
- **Власний алгоритм.** Масив ділиться на K частин; кожну рахує окремий std::thread послідовним count_if; часткові лічильники складає std::reduce.

2. Бібліотечний count_if: без політики та з політиками

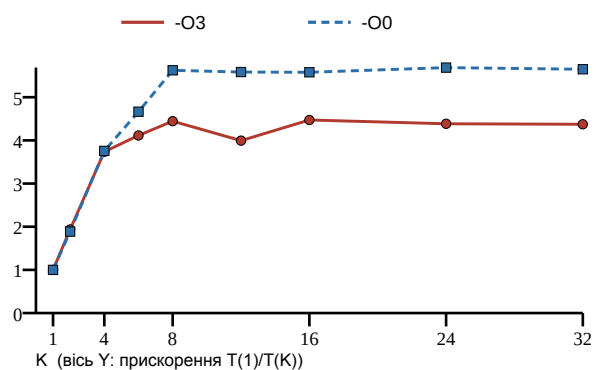
N	предикат	none	seq	par	par_unseq
100 000	even	0.127	0.125	0.047	0.031
100 000	prime	3.578	3.277	0.689	0.615
1 000 000	even	0.937	0.948	0.171	0.190
1 000 000	prime	26.816	26.693	5.925	5.959
10 000 000	even	9.385	9.576	1.604	1.764
10 000 000	prime	274.383	275.414	58.289	58.404

Таблиця 1. Час compute, мс (-O3). Результат count_if однаковий у всіх режимах.

На дорогому предикаті par/par_unseq прискорюють count_if у 4.7 раза (275 → 58 мс при N = 10 000 000): роботи на елемент багато, накладні витрати розподілу малі. На дешевому предикаті виграш є (6.0 раза), але менший і нестабільніший, бо самого обчислення мало; на малих N час (соті частки мс) порівнянний із шумом. «Без політики» та seq однопотоків й відрізняються лише в межах похибки. Таймер total для even (296 мс проти compute 9.4 мс) майже весь складається з читання текстового файлу, тому для порівняння політик використано compute.

3. Власний паралельний алгоритм (isPrime, N = 10 000 000)

K	-O3, мс	прискор.	-O0, мс	прискор.
1	266.2	1.00	717.9	1.00
2	137.5	1.94	380.4	1.89
4	71.2	3.74	191.1	3.76
6	64.7	4.11	153.9	4.66
8	59.9	4.45	127.7	5.62
12	66.6	3.99	128.6	5.58
16	59.5	4.47	128.7	5.58
24	60.7	4.38	126.3	5.69
32	60.9	4.37	127.1	5.65



Таблиця 2 і рисунок. Залежність часу від K.

Найкраще K. Формальний мінімум: K = 16 (59.5 мс) на -O3 і K = 24 (126.3 мс) на -O0, тобто 2 і 3 × hardware_concurrency (8). Але вже при K = 8 час лише на 1% і 1% більший, а при K ≥ 8 він коливається в межах 60-67 мс (-O3) і 126-129 мс (-O0): положення мінімуму всередині цього плато випадкове. Практично оптимальне K близьке до кількості апаратних потоків (8); більші K виграшу не дають.

Закон зміни часу. При малих K час падає приблизно як $T(1)/K$ (прискорення 1.94 при K = 2 і 3.74 при K = 4), далі виграш слабшає і на плато прискорення становить 4.4-4.5, тобто менше за 8 (ймовірно, ядра нерівноцінні, а розбиття додає накладні витрати). Подальше збільшення K час майже не змінює: вартість створення потоків мала порівняно з роботою частин.

4. Вплив рівня оптимізації

режим (isPrime, N = 10 000 000)	-O0, мс	-O3, мс	прискорення
none	711.3	274.4	2.59 ×
seq	710.9	275.4	2.58 ×
par	124.3	58.3	2.13 ×
par_unseq	132.5	58.4	2.27 ×
власний, K = 8	127.7	59.9	2.13 ×

Таблиця 3. Час compute залежно від рівня оптимізації.

Лише рівень оптимізації прискорює код у 2.1-2.6 раза без жодних змін і потоків. Тому політики виконання порівнювалися в межах одного рівня оптимізації: інакше «виграш від паралелізму» частково був би виграшем від оптимізації.

5. Висновки

- Політики par/par_unseq суттєво прискорюють count_if лише для важкого предиката (4.7 раза проти 6.0 раза для дешевого); «без політики» і seq еквівалентні.
- Власний алгоритм поводить ся як par: $T(K) \approx T(1)/K$ при малих K, далі плато; практично найкраще $K \approx \text{hardware_concurrency}$ (8), формальне K з мінімальним часом усередині плато нестабільне.
- Оптимізація -O3 дає 2.1-2.6 раза - співмірно з паралелізмом, тому порівнювати треба в межах одного рівня оптимізації.